PaperLens: Evidence-Grounded and Reproducibility-Aware Analysis of Research Papers with a Free-Tier LLM

Author One, Author Two, Author Three, Author Four
Department of Computer Engineering
Sardar Patel Institute of Technology, Mumbai, India
{first.last}@spit.ac.in

***Abstract*—Large language models (LLMs) summarise research papers fluently, but the facts they report come without evidence, and in research an invented number is worse than none. We present PaperLens, a system that turns a paper into a structured record in which every item carries the verbatim sentence it came from; deterministic code locates that sentence in the PDF, highlights it, and checks that it supports the item. On top of this grounded extraction, PaperLens scores reproducibility against a 21-item checklist, verifies numeric claims against the paper's own results tables, and compares the paper's hyperparameters with its released code by static analysis. One rule governs the design: *the LLM extracts and Python decides*, so every score, difference and verdict is computed deterministically, and the whole system runs on the free tier of a commercial LLM API. On a new gold set of 50 papers from nine machine-learning areas (2,796 items), extraction reaches a core-field F1 of 0.60 with precision of 0.81–0.94 per field; 90% of extracted items are verified against the paper and 1% are hallucinated. A manual audit of all extracted items absent from the gold set finds only 4% wrong, showing that the scores are lower bounds. Requiring a located, supporting quote drives hallucinations to zero at a moderate cost in recall, and recall on appendix hyperparameters is the main open problem.**

***Index Terms*—scientific information extraction, attribution, grounding, reproducibility, large language models, claim verification**

I. Introduction

Much of the time spent reading a paper goes into finding the same facts: datasets, baselines, metrics, hyperparameters and compute. Judging whether the work can be reproduced is harder still. No paper in a survey of AI research documented every factor needed for reproduction [1]; independent re-implementation of 255 papers succeeded only about two times in three [2]; and released code often uses settings other than those reported [3], [4].

LLMs are an obvious aid, but their output is unattributed and they hallucinate facts [5]. Attribution research shows that generated statements must be checked against an identified source to be trusted [6], [7]. Where a wrong learning rate silently ruins a replication, unverifiable output is of little value.

PaperLens is built around one principle: **the LLM extracts, and Python decides**. The LLM only reads text and returns schema-validated items, each with a verbatim quote. Deterministic code does everything else: it locates every quote in the PDF, scores reproducibility, matches claims to table cells and recomputes their deltas, compares the paper with its code, and issues every verdict. Because the system runs on the free tier of the Gemini API [8], each design choice also minimises LLM calls (about six per paper). Our contributions are:

1) A grounded extraction pipeline in which every item carries evidence that is located in the PDF and checked for support; unverifiable items are labelled, never dropped.
2) Three deterministic checks on top of it: a reproducibility scorecard, a claim verifier that recomputes claimed differences from parsed tables, and a static paper-versus-code comparison that never executes repository code.
3) A 50-paper, 2,796-item gold set across nine ML areas with a documented matching scheme, a manual audit of unmatched predictions, and LLM-free ablations of evidence policy, parser and retrieval.

II. Related Work

Scientific information extraction. SciERC [9] and SciREX [10] annotate entities and task–dataset–metric relations; Hou et al. [11] and AxCell [12] extract leaderboard results. These systems rely on trained encoders such as SciBERT [13], and Dagdelen et al. fine-tune LLMs for structured extraction [14]. PaperLens uses a general LLM without fine-tuning and makes evidence a checked, first-class output.

Parsing. GROBID converts PDFs to structured TEI with coordinates [15] and underlies S2ORC [16]; Nougat and Docling offer vision-based alternatives [17], [18], and table recognition has dedicated benchmarks [19], [20].

Attribution and question answering. RAG conditions generation on retrieved passages [21]; ALCE and AIS evaluate whether statements are attributable to sources [6], [7]; FActScore and SelfCheckGPT detect unsupported facts [22], [23]. Qasper anchors questions in papers [24], PaperQA and OpenScholar answer with citations over large collections [25], [26], and ScholarPhi and the Semantic Reader ground definitions inside the reading interface [27], [28].

Verification, reproducibility and reviewing. SciFact, TabFact and SciTab verify claims against abstracts and tables, and show that table reasoning remains hard for LLMs [29]–[31]. SUPER, CORE-Bench and PaperBench ask agents to run or replicate research code [32]–[34], at hours of compute per paper. LLM reviewers overlap with human reviewers in coverage but are unreliable for scoring [35], [36].

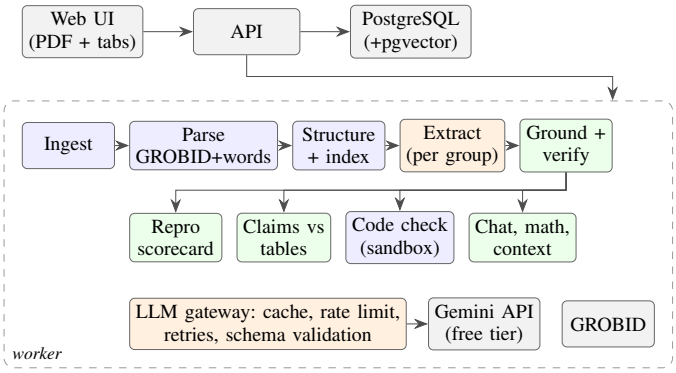


Fig. 1. Architecture. Orange stages call the LLM, blue stages are pure Python, and green stages make at most one LLM call whose output Python checks.

Positioning. Table I compares design properties. Each line of work covers part of the problem; PaperLens is, to our knowledge, the only one that fills a full extraction schema including hyperparameters, checks a quote for every item in code, leaves numeric verdicts to deterministic code, and adds reproducibility, claim and code checks without fine-tuning or code execution. The comparison is of design, not accuracy: the systems address different tasks, so their scores are not comparable.

III. System Overview

Fig. 1 shows the architecture. A web interface accepts a PDF or a link (arXiv, DOI, OpenReview, ACL Anthology); a worker runs the pipeline and streams progress. Every LLM call passes through one gateway that caches responses, rate-limits per model, retries with backoff, falls back to a backup model when one is overloaded, validates the output against a schema with one repair attempt, and logs the model and prompt version. Results are keyed by (paper, stage, prompt version, model), so stages are idempotent and cached re-runs are free.

IV. Method

A. Parsing

GROBID supplies sections, sentences with coordinates, references and resolved in-text citations; all coordinates are kept in PDF points so any span can later be highlighted. Because GROBID's table grids were often wrong or misplaced, PaperLens locates each table by its own caption ("Table 3:") and rebuilds the grid from the PDF's word positions, merging wrapped cells and parsing values such as 24.3 ± 0.2. A grid that fails a sanity check is dropped: no grid is better than a wrong one.

B. Grounded extraction

Sections are grouped into front matter, method, experiments and closing, and each group is sent in one schema-constrained call with its paragraphs labelled by chunk id. For each item the LLM returns a value, a verbatim quote and the chunk id. Python then locates each quote q in the cited chunk c and, failing that, in the whole paper D, using the Levenshtein-based partial-ratio similarity s [37] of normalised strings:

$$\hat{b}(q) = \arg\max_{b \in \{c\} \cup D} s(q, b), \qquad \text{located} \iff s\big(q, \hat{b}(q)\big) \geq \tau, \tag{1}$$

with $\tau = 90$. Located quotes are mapped to sentence boxes; a quoted table row highlights that row. One batched call then labels each located quote as *verified*, *partial* or *unsupported*; an unfound quote is *unlocated*. Such items are kept and badged, so users see them and the evaluation counts them. Code links are found by rules, not by the LLM.

C. Reproducibility scorecard

A 21-item checklist derived from the NeurIPS checklist [3] is split into five groups g with weights W_g: code (0.25), data (0.20), experimental setup (0.25), compute (0.10) and statistical reporting (0.20). Rules (regular expressions, document structure, repository metadata) decide items first; one LLM call judges the rest, and each judgement must quote the paper. An LLM "present" whose quote is not confirmed is capped at "partial". With $p(\text{present}) = 1$, $p(\text{partial}) = 0.5$, $p(\text{missing}) = 0$ and item weights w_i over the scored items S_g,

$$\text{score} = 100 \cdot \frac{\sum_g W_g \, \sigma_g}{\sum_g W_g}, \qquad \sigma_g = \frac{\sum_{i \in S_g} w_i \, p(s_i)}{\sum_{i \in S_g} w_i}. \tag{2}$$

D. Claim verification

For each quantitative claim the LLM only copies the method, baseline, metric, dataset and claimed difference δ^* as written. Python finds the candidate cells x_m and x_b in the parsed results tables, recomputes $\hat{\delta} = x_m - x_b$ (or a ratio), and labels the claim *supported* if $|\hat{\delta} - \delta^*| \leq \epsilon$ ($\epsilon = 0.15$). *Contradicted* requires exactly one unambiguous pair of cells; ambiguous or missing matches yield *unsupported*, so a parsing error cannot make a paper look wrong.

E. Paper versus code

Repository links are ranked from the paper text, separating the authors' own code from libraries the work is "based on". The repository is cloned shallowly with hooks, symlinks and local protocols disabled and a size cap, and a parse-only scanner reads Python ASTs, YAML, JSON, TOML and README flags inside a sandbox; repository code is never executed. Names are aligned by a synonym table (`lr` ↔ `learning_rate`) and values compared after normalisation ($3\text{e}{-4} = 0.0003$).

F. Chat, notation and review

Chat retrieves paragraphs with BM25 [38] and dense embeddings [39], [40] fused by reciprocal rank fusion [41]; Python removes citation markers the model invents and refuses uncited answers, or answers nothing when retrieval finds nothing relevant. A notation glossary is built from the arXiv LaTeX source (read, never compiled), with each symbol's meaning grounded in a sentence; key references are those cited most often in the text, optionally enriched by Semantic

TABLE I
DESIGN COMPARISON WITH REPRESENTATIVE FRAMEWORKS (✓ CORE FEATURE, (✓) PARTIAL, – ABSENT). THE LAST COLUMN STATES WHAT EACH NEEDS TO RUN.

Framework	Schema extraction (incl. HPs)	Quote per item	Quote checked in code	Highlight in PDF	Numeric verdicts in code	Repro. check	Claims vs tables	Paper vs code	Halluc. measured	No fine-tuning	Needs
SciREX [10]	(✓)	(✓)	–	–	–	–	–	–	–	–	trained model
AxCell [12]	(✓)	–	–	–	–	–	(✓)	–	–	–	trained model
LLM-NERRE [14]	✓	–	–	–	–	–	–	–	–	–	fine-tuned LLM
PaperQA [25]	–	✓	(✓)	–	–	–	–	–	(✓)	✓	LLM API
OpenScholar [26]	–	✓	(✓)	–	–	–	–	–	(✓)	(✓)	LLM + paper datastore
ScholarPhi / Semantic Reader [27], [28]	–	✓	(✓)	✓	–	–	–	–	–	–	trained detectors
SciFact, SciTab [29], [31]	–	(✓)	–	–	–	–	✓	–	–	(✓)	model verdict
SUPER, CORE-Bench, PaperBench [32]–[34]	–	–	–	–	(✓)	✓	–	(✓)	–	✓	agent runs code (hours)
LLM reviewers [35]	–	–	–	–	–	(✓)	–	–	–	✓	LLM API
General LLM chat over a PDF	(✓)	(✓)	–	–	–	(✓)	(✓)	–	–	✓	LLM API
PaperLens (ours)	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	free-tier API, CPU

TABLE II
GOLD SET COMPOSITION (* = DEVELOPMENT SET).

Area	n	Papers
NLP	16	Bahdanau*, Transformer*, BERT*, RoBERTa*, DistilBERT*, LoRA*, T5, ELECTRA, SimCSE, RAG, chain-of-thought, ReAct, LLaMA, DPO, QLoRA, DeepSeekMath
CV	13	VGG*, ResNet*, SimCLR*, ViT*, Faster R-CNN, U-Net, Swin, MAE, ConvNeXt, DINOv2, SAM, ViT registers, SigLIP
ML	9	BatchNorm, AdamW, PGD, Lottery Ticket, scaling laws, Chinchilla, FlashAttention, Mamba, KAN
Other	12	latent diffusion, DiT, flow matching, LLaVA, GCN, GAT, PPO, SAC, wav2vec 2.0, Whisper, PatchTST, iTransformer

Scholar [42]. A draft review lists quoted strengths, weaknesses and questions, keeps facts computed in Python separate, and gives no score.

V. EXPERIMENTAL SETUP

Gold set. We annotated 50 arXiv papers from 2014–2024 (Table II). Ten *development* papers were used while building the system: Bahdanau et al. [43], VGG [44], ResNet [45], Transformer [46], BERT [47], RoBERTa [48], DistilBERT [49], SimCLR [50], ViT [51] and LoRA [52]. Forty *held-out* papers were added afterwards, balanced over nine areas and varied in layout: appendix hyperparameter tables, prompting papers with few settings, theory-heavy papers and papers without released code. The 2,796 gold items cover task, datasets, baselines, metrics, hyperparameters, compute and code URLs, defined in an annotation guide. The reference annotations were produced by an LLM (Claude) reading each paper's full text under the guide, then adjudicated once against PaperLens' output: 48 omitted items that the papers state were added, no alias was written in the system's wording, and nothing was removed. Every change is logged.

Matching and metrics. Text values are normalised (Unicode NFKD, number words to digits). A prediction matches a gold item when the two are nearly equal (fuzzy ratio ≥ 88 with equal digit sequences, so CIFAR-10 never matches CIFAR-100), or when one contains the other and the remainder is filler. Hyperparameters must agree on the name (through a synonym table) and on the parsed value ($6\% = 0.06$). For predicted items P and gold items G of a field,

$$\mathrm{P} = \frac{|\{p \in P : \exists g,\, p \sim g\}|}{|P|}, \quad \mathrm{R} = \frac{|\{g \in G : \exists p,\, p \sim g\}|}{|G|}, \tag{3}$$

and F1 is their harmonic mean, micro-averaged over papers; the *core-field F1* is the mean over task, datasets, baselines, metrics and hyperparameters. The *hallucination rate* is the share of extracted items that are unlocated, unsupported or without evidence; *numeric accuracy* is the share of extracted hyperparameters with a gold name whose value is correct; *hyperparameter coverage* is the share of gold (paper, name) pairs with a correct value extracted.

Configuration. Extraction uses Gemini Flash-Lite on the free tier with GROBID 0.8.1 and the thresholds above; all numbers come from a single run over the 50 papers, and all tables and figures are regenerated by one command from the stored outputs.

VI. RESULTS

A. Extraction quality

Table III reports per-field scores. Precision on the core fields is 0.81–0.94; recall is the bottleneck, above all for hyperparameters (0.27) and tasks (0.37), giving a core-field F1 of 0.60. Of the extracted hyperparameters whose name matched a gold name, 91% carried the right value. 90% of all 2,283 extracted items are verified against the paper, and 1% are unlocated or unsupported.

Table IV splits the results. The development papers reach 0.72 and the held-out papers 0.57: the prompts were tuned on the former, and several held-out papers state settings per model size in appendix tables (LLaMA, Whisper, T5). Graph papers, with a single table of settings, are easiest; RL and generative-model papers, whose settings sit in long appendix tables, reach only 7% hyperparameter coverage. Areas with two or three papers are indicative only. Fig. 2 shows every paper.

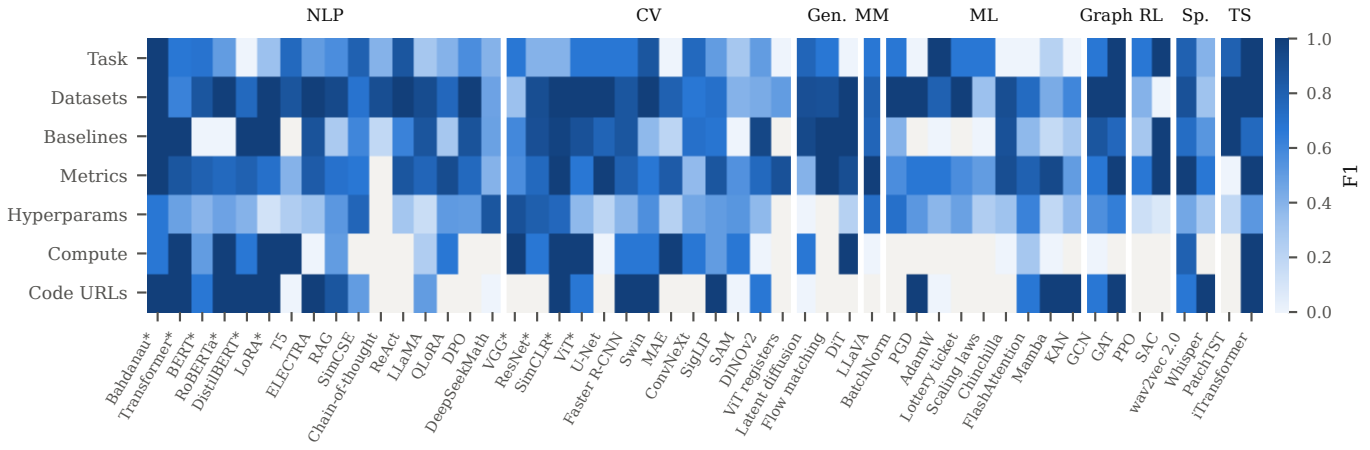


Fig. 2. Per-paper F1 by field, grouped by area (* development set; grey: no gold item in that field). Gen. = generative, MM = multimodal, Sp. = speech, TS = time series.

TABLE III
EXTRACTION RESULTS PER FIELD (50 PAPERS).

Field	Gold	Pred	P	R	F1	$F1_{macro}$
Task	172	78	0.81	0.37	0.51	0.53
Datasets	490	320	0.94	0.63	**0.75**	0.79
Baselines	472	306	0.84	0.54	0.66	0.62
Metrics	239	171	0.89	0.62	0.73	0.74
Hyperparameters	1310	407	0.83	0.27	0.41	0.43
Compute	71	70	0.54	0.51	0.52	0.59
Code URLs	42	68	0.44	0.69	0.54	0.73
Core-field F1					**0.60**	
Hyperparameter coverage / numeric accuracy					30% / 91%	
Verified evidence / hallucination rate					90% / 1%	

TABLE IV
RESULTS BY PAPER SET AND AREA (50 PAPERS).

Group	n	Core F1	HP coverage	Verified
Development set	10	0.72	40%	88%
Held-out set	40	0.57	28%	91%
NLP	16	0.61	32%	90%
CV	13	0.60	36%	88%
ML	9	0.51	32%	90%
Generative	3	0.56	7%	96%
Graph	2	0.82	59%	94%
RL	2	0.43	7%	82%
Speech	2	0.62	24%	95%
Time series	2	0.68	26%	93%
Multimodal	1	0.79	50%	94%

B. Evidence policy (ablation A3)

Because every item carries an evidence status, the same extraction can be scored under stricter policies without new LLM calls (Fig. 3). On the development set with an earlier pipeline version, which left more quotes unlocated, requiring a located quote cut the hallucination rate from 10% to 2%, and additionally requiring support cut it to 0%, with precision unchanged (0.96–0.97) and recall falling from 0.54 to 0.43. On the 50-paper run nearly every quote is located, so the strictest policy only removes the last 1% at a recall cost of 0.03. We therefore keep every item in the interface with its status badge rather than filtering.

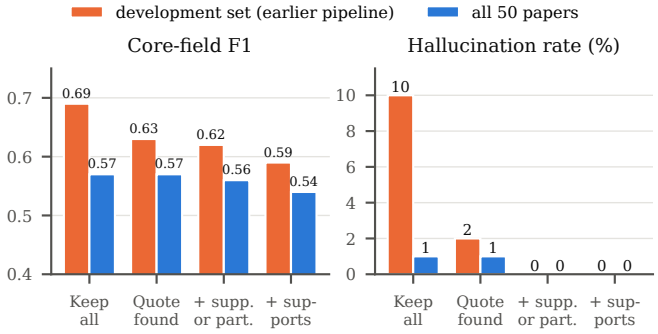


Fig. 3. Ablation A3: core-field F1, micro over all core items (left), and hallucination rate (right) under four evidence policies, for the development set (earlier pipeline) and all 50 papers.

C. Parser (A1) and retrieval (A6)

Fig. 4 measures how many gold values survive parsing at all, an upper bound on recall. Raw PyMuPDF text keeps more values than GROBID (90% vs 83%), mostly numbers in tables, footnotes and appendix lists, but GROBID recovers the structure the pipeline depends on (1,520 vs 1,165 section headings, 3,026 vs 2,394 references, 580 vs 243 equations). For hyperparameters the gap is widest: 33% of gold values never reach the extractor through GROBID's text. With a ceiling of 67%, the observed 30% coverage means the extractor recovers less than half of what the parse makes available. For chat, recall@5 on 2,086 question–item pairs is 45% for BM25, 36% for dense retrieval and 45% for the hybrid, which helps on datasets, metrics and hyperparameters but loses on task questions (45% vs 53%); keeping BM25 in the fusion is therefore essential.

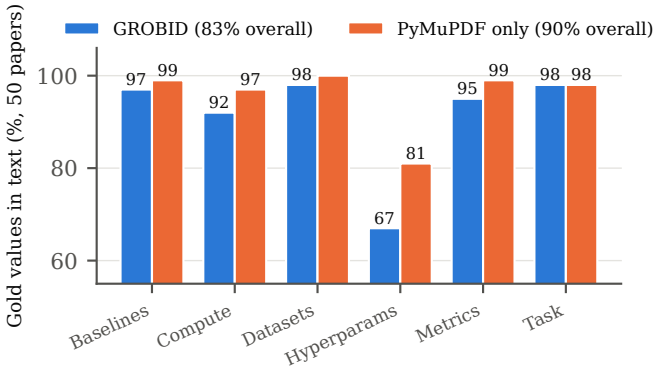


Fig. 4. Ablation A1: gold values present in the parsed text, by field (50 papers).

D. Downstream checks

On the development set, the reproducibility scorecard agreed with human annotations on 158 of 210 checklist decisions (75%, Cohen's $\kappa = 0.57$ [53]); the main disagreements concern training time and re-run baselines. Repository discovery found the official code for all eight development papers that release it. As an example of the code check, BERT [47] reports pre-training with learning rate 10^{-4}, batch size 256 and 1M steps, while the defaults of the official `run_pretraining.py` are 5×10^{-5}, 32 and 100,000; PaperLens reports the mismatch with the paper's sentence beside the code location. The defaults are legitimate command-line values, but running the script as is would not reproduce the paper.

VII. Error Analysis

Unmatched predictions. We read each of the 246 extracted items of the held-out papers that matched no gold item against the paper (Table V). Only 10 (4%) are wrong: model families listed as datasets, a method listed as its own baseline, two metrics and two values the paper does not state, and one code link absent from the text. Half (127) are correct items the strict matcher misses, either through hyperparameter names ("lambda" vs `loss_weight_lambda`) or through values garbled by superscripts in the PDF text layer ("3.0eˆ−ˆ4", "2 24" for 2^{24}). A further 31 were gold omissions, since added, and 78 are out of scope, mostly third-party links and compute descriptions without hardware or time. Precision in Table III is thus a lower bound.

Missed gold items. The run missed 1,602 of 2,796 gold items, 956 of them hyperparameters. Four causes recur. (i) *Per-size appendix settings*: LLaMA, Whisper and T5 list dimensions, heads, layers and learning rates for every model size, and the extraction usually returns one size. (ii) *Narrow tasks*: the model names one or two tasks where the experiments address several (e.g. SAM's edge detection and object proposals). (iii) *Long benchmark suites*: DINOv2 misses 29 of 42 datasets and Whisper 17 of 21, while LLaMA, whose benchmarks appear in running text, misses 4 of 27. (iv) *Parsing*: 17% of all gold values and 33% of hyperparameter values are absent from GROBID's text (A1).

TABLE V
Manual audit of the 246 unmatched predictions (held-out papers).

Field	Correct, matcher missed	Gold omission	Out of scope	Wrong
Task / datasets	14	6	11	4
Baselines / metrics	48	4	9	3
Hyperparameters	54	19	2	2
Compute / code URLs	11	2	56	1
All	**127**	31	78	**10**

VIII. Discussion and Limitations

Precision by construction, recall by effort. Requiring a quote for every item and checking it in code yields high precision and a hallucination rate that can be driven to zero. Recall instead depends on the model reading every table and appendix, and on the parser keeping them; table-aware extraction of per-size settings and a parse that retains appendix tables are the most promising next steps.

Deterministic decisions. Scores, differences, verdicts and code comparisons are computed in Python, so they are reproducible, unit-tested and conservative: an ambiguous table match yields "unsupported", never "contradicted".

Limitations and threats to validity. The gold set has a single annotator, an LLM, adjudicated once against this system's output, and the audit was done the same way; where the annotating LLM and Gemini share blind spots, recall may be overestimated, while the strict matcher underestimates precision. Areas outside NLP, CV and ML have one to three papers. All results come from one run of one model on well-known papers that the model may have seen in training. The scorecard's agreement with human judgements is below our target of 80%, and ablations that require additional LLM runs (extraction granularity, model choice, extraction without quotes) are left to future work.

Ethics. Paper text is sent to a free-tier API whose inputs may be used by the provider, so the interface warns against uploading unpublished work. Repository code is never executed, and the draft review gives no score, consistent with evidence that LLM reviews should support rather than replace human reviewers [35].

IX. Conclusion

PaperLens shows that a free-tier LLM can support careful paper reading when it is asked only to extract and quote, while deterministic code locates every quote and makes every decision. On 50 papers from nine areas it extracts with high precision (4% of unmatched predictions are wrong), 91% numeric accuracy and 1% hallucinated items, each item traceable to a highlighted sentence. Closing the recall gap on appendix tables, and adding independent human annotation, are the next steps.

ACKNOWLEDGMENT

The authors thank their project guide for feedback.

REFERENCES

[1] O. E. Gundersen and S. Kjensmo, "State of the art: Reproducibility in artificial intelligence," in *Proc. AAAI Conf. Artificial Intelligence*, 2018, pp. 1644–1651.

[2] E. Raff, "A step toward quantifying independently reproducible machine learning research," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[3] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché Buc, E. Fox, and H. Larochelle, "Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program)," *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021.

[4] C. Collberg and T. A. Proebsting, "Repeatability in computer systems research," *Communications of the ACM*, vol. 59, no. 3, 2016.

[5] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, 2023.

[6] H. Rashkin, V. Nikolaev, M. Lamm, L. Aroyo, M. Collins, D. Das, S. Petrov, G. S. Tomar, I. Turc, and D. Reitter, "Measuring attribution in natural language generation models," *Computational Linguistics*, vol. 49, no. 4, pp. 777–840, 2023.

[7] T. Gao, H. Yen, J. Yu, and D. Chen, "Enabling large language models to generate text with citations," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 6465–6488.

[8] Gemini Team, Google, "Gemini: A family of highly capable multimodal models," arXiv:2312.11805, 2023.

[9] Y. Luan, L. He, M. Ostendorf, and H. Hajishirzi, "Multi-task identification of entities, relations, and coreference for scientific knowledge graph construction," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2018.

[10] S. Jain, M. van Zuylen, H. Hajishirzi, and I. Beltagy, "SciREX: A challenge dataset for document-level information extraction," in *Proc. Annu. Meeting Assoc. Computational Linguistics (ACL)*, 2020.

[11] Y. Hou, C. Jochim, M. Gleize, F. Bonin, and D. Ganguly, "Identification of tasks, datasets, evaluation metrics, and numeric scores for scientific leaderboards construction," in *Proc. Annu. Meeting Assoc. Computational Linguistics (ACL)*, 2019, pp. 5203–5213.

[12] M. Kardas, P. Czapla, P. Stenetorp, S. Ruder, S. Riedel, R. Taylor, and R. Stojnic, "AxCell: Automatic extraction of results from machine learning papers," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 8580–8594.

[13] I. Beltagy, K. Lo, and A. Cohan, "SciBERT: A pretrained language model for scientific text," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019.

[14] J. Dagdelen, A. Dunn, S. Lee, N. Walker, A. S. Rosen, G. Ceder, K. A. Persson, and A. Jain, "Structured information extraction from scientific text with large language models," *Nature Communications*, vol. 15, p. 1418, 2024.

[15] P. Lopez, "GROBID: Combining automatic bibliographic data recognition and term extraction for scholarship publications," in *Research and Advanced Technology for Digital Libraries (ECDL)*, ser. Lecture Notes in Computer Science, vol. 5714, 2009.

[16] K. Lo, L. L. Wang, M. Neumann, R. Kinney, and D. S. Weld, "S2ORC: The semantic scholar open research corpus," in *Proc. Annu. Meeting Assoc. Computational Linguistics (ACL)*, 2020.

[17] L. Blecher, G. Cucurull, T. Scialom, and R. Stojnic, "Nougat: Neural optical understanding for academic documents," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[18] C. Auer *et al.*, "Docling technical report," arXiv:2408.09869, 2024.

[19] X. Zhong, E. ShafieiBavani, and A. J. Yepes, "Image-based table recognition: Data, model, and evaluation," in *Proc. European Conf. Computer Vision (ECCV)*, 2020.

[20] B. Smock, R. Pesala, and R. Abraham, "PubTables-1M: Towards comprehensive table extraction from unstructured documents," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 4634–4642.

[21] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[22] S. Min, K. Krishna, X. Lyu, M. Lewis, W. tau Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[23] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[24] P. Dasigi, K. Lo, I. Beltagy, A. Cohan, N. A. Smith, and M. Gardner, "A dataset of information-seeking questions and answers anchored in research papers," in *Proc. Conf. North American Chapter Assoc. Computational Linguistics (NAACL-HLT)*, 2021.

[25] J. Lála *et al.*, "PaperQA: Retrieval-augmented generative agent for scientific research," arXiv:2312.07559, 2023.

[26] A. Asai *et al.*, "Synthesizing scientific literature with retrieval-augmented language models," *Nature*, 2026, doi:10.1038/s41586-025-10072-4.

[27] A. Head, K. Lo, D. Kang, R. Fok, S. Skjonsberg, D. S. Weld, and M. A. Hearst, "Augmenting scientific papers with just-in-time, position-sensitive definitions of terms and symbols," in *Proc. CHI Conf. Human Factors in Computing Systems*, 2021.

[28] K. Lo, J. C. Chang, A. Head, J. Bragg, A. X. Zhang, C. Trier *et al.*, "The semantic reader project: Augmenting scholarly documents through AI-powered interactive reading interfaces," *Communications of the ACM*, vol. 67, no. 10, 2024.

[29] D. Wadden, S. Lin, K. Lo, L. L. Wang, M. van Zuylen, A. Cohan, and H. Hajishirzi, "Fact or fiction: Verifying scientific claims," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[30] W. Chen, H. Wang, J. Chen, Y. Zhang, H. Wang, S. Li, X. Zhou, and W. Y. Wang, "TabFact: A large-scale dataset for table-based fact verification," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2020.

[31] X. Lu, L. Pan, Q. Liu, P. Nakov, and M.-Y. Kan, "SCITAB: A challenging benchmark for compositional reasoning and claim verification on scientific tables," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 7787–7813.

[32] B. Bogin, K. Yang, S. Gupta, K. Richardson, E. Bransom, P. Clark, A. Sabharwal, and T. Khot, "SUPER: Evaluating agents on setting up and executing tasks from research repositories," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2024, pp. 12 622–12 645.

[33] Z. S. Siegel, S. Kapoor, N. Nadgir, B. Stroebl, and A. Narayanan, "CORE-Bench: Fostering the credibility of published research through a computational reproducibility agent benchmark," *Transactions on Machine Learning Research*, 2024.

[34] G. Starace *et al.*, "PaperBench: Evaluating AI's ability to replicate AI research," in *Proc. Int. Conf. Machine Learning (ICML)*, ser. PMLR, vol. 267, 2025.

[35] W. Liang, Y. Zhang *et al.*, "Can large language models provide useful feedback on research papers? a large-scale empirical analysis," *NEJM AI*, vol. 1, no. 8, 2024.

[36] R. Zhou, L. Chen, and K. Yu, "Is LLM a reliable reviewer? a comprehensive evaluation of LLM on automatic paper reviewing tasks," in *Proc. Joint Int. Conf. Computational Linguistics, Language Resources and Evaluation (LREC-COLING)*, 2024, pp. 9340–9351.

[37] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[38] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[39] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019.

[40] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih, "Dense passage retrieval for open-domain question answering," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[41] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2009.
[42] M. Valenzuela, V. Ha, and O. Etzioni, "Identifying meaningful citations," in *AAAI Workshop on Scholarly Big Data: AI Perspectives, Challenges, and Ideas*, 2015.
[43] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.
[44] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.
[45] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.
[46] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
[47] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North American Chapter Assoc. Computational Linguistics (NAACL-HLT)*, 2019.
[48] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," arXiv:1907.11692, 2019.
[49] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," arXiv:1910.01108, 2019.
[50] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. Int. Conf. Machine Learning (ICML)*, 2020.
[51] A. Dosovitskiy *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.
[52] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.
[53] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.